ELSEVIER

Int. J. Production Economics 76 (2002) 265–279

international journal of
production
economics

www.elsevier.com/locate/dsw

Parallel machine scheduling to minimize total tardiness

Farouk Yalaoui*, Chengbin Chu

Université de Technologie de Troyes, Industrial Systems Optimization Laboratory, 12 rue Marie Curie, St., BP-2060, 10010 Troyes Cedex, France

Received 19 July 2000; accepted 11 June 2001

Abstract

This paper addresses, using an exact method, the identical parallel machine scheduling problem to minimize total tardiness. In this problem, a set of jobs has to be scheduled, without preemption, on identical parallel machines. Each machine is able to treat only one job at a time. In order to prune the search tree, dominance properties are proved and lower and upper bounding schemes are proposed. A branch and bound (BAB) algorithm is developed taking into account these theoretical properties, lower and upper bounds. This BAB algorithm has been tested on a large set of randomly generated medium-sized problems. Computational results demonstrate the effectiveness of our approach over existing methods in the literature.

Keywords: Parallel machine scheduling; Total tardiness; Dominance properties; Lower bound; Branch and bound

1. Introduction

This paper deals with the identical parallel machine scheduling problem to minimize total tardiness. Minimizing total tardiness is among the most interesting criteria for production systems, especially in the current situation where competition is becoming more and more intensive. This paper aims at developing an exact method to solve this problem.

This problem can be described as follows. A set of N jobs is to be scheduled without preemption on M identical parallel machines, which are assumed to be continuously available. Each machine can process at most one job at a time. The jobs are assumed to be all available at time zero. With each job i, are associated a processing time p_i and a due date d_i. The objective is to determine a schedule such that the total tardiness of jobs is minimized. A job is said to be ardy if its completion time C_i is greater than its due date d_i and its tardiness is measured by $C_i - d_i$. If a job is not tardy, its tardiness is set to 0, without loss of generality. Therefore, the tardiness of job i is given by $\max(C_i - d_i, 0)$.

According to Koulamas [1], the problem considered here is at least binary NP-hard since Du and Leung [2] showed that the problem with $M = 1$ is binary NP-hard. Lenstra et al. [3] also showed that the problem with two machines was binary NP-hard. Nevertheless the exact complexity of this problem remains open. The literature on this problem with $M > 1$, is relatively limited compared to its single machine counterpart.

*Corresponding author. Tel.: +33-325715626; fax: +33-325715649.
E-mail address: yalaoui@univ-troyes.fr (F. Yalaoui).

0925-5273/02/$ - see front matter
PII: S0925-5273(01)00175-X

This remark was already made by Koulamas in 1994 [1] in a very interesting work where a state of the art on tardiness minimization problems was presented.

The development of heuristics has been carried out in some research works. Montagne [4] introduced a rule called Montagne's ratio method (MRM) for total weighted tardiness minimization. The MRM rule schedules jobs in increasing order of the ratio $p_i/(N\times\bar{p}-d_i)$ with $\bar{p}=\sum_{i=1}^{N}p_i/N$ being the average processing time. In 1971, Wilkerson and Irwin [5] developed a method, noted WI, based on neighbor search to minimize the average tardiness on a machine. This method was formally presented by Baker [6] and used to solve the parallel machines case. It consists of ranking the jobs according to the earliest due date (EDD) rule, of selecting an unscheduled job, with the earliest due date, and assign it to a machine according to the following rule: if the job selected can be added after a partial schedule without being late, then the job is assigned on the machine such that it can be completed as closely as possible to its due date, otherwise it is assigned to the earliest available machine. According to Baker's experimentation, the WI gave results about 1.6% from optimal solutions for small-sized problems. In 1979, Dogramaci and Surkis [7], proposed a heuristic, for parallel machine problem, using list algorithms. This heuristic makes it possible to obtain three different solutions, by using three priority rules independently: shortest processing time (SPT), EDD and minimum slack (the slack of a job j is defined as $SL_j=d_j-p_j$). The best one out of these three solutions is retained. In 1991, Ho and Chang [8] built an assignment rule noted traffic priority index (TPI). This method combines the SPT and EDD rules using a new measurement called traffic congestion ratio (TCR). Then, they built heuristics for the cases with one or identical machine. These heuristics consist of building a first solution by scheduling jobs in increasing order of their priority index, then an improvement of this solution is obtained using permutation technique of WI method previously quoted. Koulamas presented several works to solve this problem. First in 1994, he developed a heuristic, called KPM, which is an extension to parallel machines of PSK heuristic of Panwalkar et al. [9], conceived initially for the single machine problem. This method gave, according to the authors, more interesting results than other heuristics previously presented. In 1997, Koulamas proposed a method called PDEC based on method of Potts and Wassenhove [10] for the single machine problem – this method is based on the combination of a decomposition procedure and the WI method. The PDEC method is a decomposition procedure coupled with PSK heuristic for parallel machine problem. It consists of bringing back the problem with parallel machines to a system of several problems with a single machine on which theoretical properties are available, where the PSK method is applicable. In addition to the PSK method, Koulamas [11] developed hybrid simulated annealing (HSA) heuristic based on simulated annealing. According to the authors these two methods are effective compared to the others. But one of the disadvantages of HSA is the relatively long completion time. We have also highest priority job first (HPJF) method of Chen et al. [12], which is an extension of the WI method supplemented by various priority rules such as: Minimum processing time first (priority = 1/processing time), Maximum processing time first (priority = processing time), Minimum deadline first (priority = 1/due date) and Maximum deadline first (priority = due date). In 1997, Alidaee and Rosa [13] proposed a heuristic that is an extension of the modified due date (MDD) method of Baker and Bertrand [14]. This last method was, according to the experimentation of authors, quite effective for the parallel machine problem.

As far as exact methods are concerned, to the best of our knowledge, only Azizoglu and Kirca [15] proposed a branch and bound (BAB) approach to handle the same problem we consider here. Nevertheless, optimal methods are available for problems very similar to our problem at hand. Among these methods, we can refer to the formulation of Lawler [16] of the parallel machine problem as a transportation problem when the job processing times are identical. In 1965, assuming that all jobs have identical due dates, Root [17] developed a method by construction. For their part, Pritsker et al. [18] in 1969, modeled with 0–1 linear programming the limited resource problem where the minimization of total tardiness on identical parallel machines is a special case. By considering identical due dates and processing times, Elmaghraby and Park in 1974 [19],

proposed a branch and bound to minimize a function of penalties related to tardiness. Three years later, this method was taken again and improved by Barnes and Brennan [20].

In addition to previous quoted works, there exist a set of studies that deal with parallel machine scheduling problem but with alternatives. For the minimization of the total weighted tardiness we have for example: Arkin and Roundy [21] or Emmons and Pinedo [22]; in the case of uniform or unspecified parallel machines or in the stochastic case of scheduling: Guinet [23] or Emmons [24] or Azizoglu and Kirca [25].

2. Theoritical properties

The problem considered in this paper is to schedule N jobs indexed from 1 to N on M identical machines indexed from 1 to M. Each job $i\,(i = 1, 2, \ldots, N)$ has a processing time p_i and a due date d_i. Before proceeding, note that a schedule corresponds to a unique permutation on the set $\{1, 2, \ldots, N\}$. In fact, for any given permutation, a corresponding schedule can be constructed by assigning the next unscheduled job in the list to the machine available the earliest, ties broken by selecting the least indexed machine. Conversely, for any given semi-active schedule, a permutation can be obtained by ordering the jobs in non decreasing starting times, ties broken in non decreasing order of machine indexes. As a consequence, a partial schedule is a permutation of a subset of $\{1, 2, \ldots, N\}$. In the remainder, the word "schedule" indifferently refers to a partial schedule or complete schedule, unless additional precision is given. The following notation will be used.

$J(K)$ set of jobs scheduled in schedule K
$J_m(K)$ set of jobs scheduled on machine m in schedule K
$\Delta_i(K)$ completion time of the job immediately preceding job i in schedule K. If job i is the first job in K, $\Delta_i(K)$ is set to 0, without loss of generality
$C_I(K)$ completion time of job i in schedule K
$T_m(K)$ total tardiness of the jobs scheduled on machine m in schedule K
$\phi_m(K)$ completion time of the last scheduled job on machine m in schedule K
$\mu(K)$ the machine available the earliest in schedule K (*the machine with the smallest* $\phi_m(K)$)
$K \circ i$ the new schedule obtained by adding job i behind K (i.e. by adding job i on the machine $\mu(K)$)
$\Omega(K, i)$ the schedule composed of $K \circ i$, completed by the partial optimal schedule of remaining jobs.

When there is no ambiguity, $J(K)$, $J_m(K)$, $\Delta_i(K)$, $C_i(K)$, $T_m(K)$, $\phi_m(K)$, and $\mu(K)$ are simplified into J, J_m, Δ_i, C_i, T_m, ϕ_m and μ, respectively.

Let $\mu'(K)$ be the machine available just after machine $\mu(K)$ and before all other machines of the system.

Based on the general properties presented in [26] and [27], we have:

Theorem 1. *There is an optimal schedule such that the number of jobs scheduled on any machine is less than or equal to* $N - M + 1$.

Corollary 1. *Any partial schedule K such that more than* $N - M + 1$ *jobs are scheduled on a machine can be discarded. In any optimal schedule resulting from K, the number of additional jobs on a machine m is at most*

$$U_m(K) = \min[N - M + 1 - |J_m(K)|, N - |J(K)|].$$

In the remainder, we denote by U^* the maximum over all $U_m(K)$'s; i.e.

$$U^* = \max_{1 \leqslant m \leqslant M} U_m(K) \tag{1}$$

and also $U = U_\mu(K) = \min[N - M + 1 - |J_\mu(K)|, N - |J(K)|]$.

2.1. Dominance properties

In this subsection, some dominance properties are proved in order to eliminate dominated partial schedules and therefore to speed up the branch and bound algorithm. For the minimization of total tardiness on identical parallel machines we obtained the following properties.

Theorem 2. *For any partial schedule K and for any job i unscheduled in K, if there is another job j unscheduled in K such that*: $p_j \leqslant p_i$, and $(p_i - p_j)(U * -1) \leqslant \min(d_i - p_i, \phi_{\mu'}) - \max(d_j - p_j, \phi_\mu)$, *then* $\Omega(K, i)$ *is dominated.*

Theorem 3. *For any partial schedule K and for any job i unscheduled in K, if there is another job j unscheduled in K such that*: $0 \leqslant p_j - p_i \leqslant \underline{p}$, $d_j \leqslant \phi_\mu + p_j \leqslant d_i$ and $(p_j - p_i)(U - 1) \leqslant \min(\phi_{\mu'}, d_i - p_i) - \phi_\mu$, *then* $\Omega(K, i)$ *is dominated, where* $\underline{p}$ *is the smallest processing time of the unscheduled jobs.*

These two properties can be proved using the same scheme. In this scheme, two cases are examined: In the first case jobs i and j are scheduled on the same machine μ whereas in the second, they are scheduled on different machines. For both cases, we check whether there is another schedule S at least as good as $\Omega(K, i)$, by interchanging the positions of jobs i and j. To simplify the notation, $\Omega(K, i)$ is noted as S. By definition, the total tardiness of schedule S is

$$T(S) = \sum_{k=1}^{M} T_k(S) = \sum_{h=1}^{N} \max(C_h(S) - d_h, 0). \tag{2}$$

In the proofs that will follow, we will consider the following notation:

"Case 1" refers to the case where jobs i and j are both scheduled on machine μ in S. "Case 2" refers to the other case. $A_{i \to j}$ is the set of jobs scheduled between jobs i and j on machine μ in S. A_k is the set of jobs scheduled after job k on the same machine in S.

Proof of Theorem 2. The conditions of the theorem imply:

$$\min(d_i - p_i, \phi_{\mu'}) - \max(d_j - p_j, \phi_\mu) \geqslant 0.$$

Therefore

$$d_i - p_i \geqslant d_j - p_j \Rightarrow d_i \geqslant d_j + (p_i - p_j) \geqslant d_j,$$

$$d_i - p_i \geqslant \phi_\mu \Rightarrow d_i \geqslant \phi_\mu + p_i.$$

Case 1: if jobs i and j are both scheduled on machine μ, construct a new schedule S' by permuting jobs i and j. Let t be the starting time of job j in S.

In this new schedule, the completion time and hence the tardiness of any job between i and j is decreased. Therefore

$$\begin{aligned} T(S') - T(S) &\leqslant T_i(S') + T_j(S') - T_i(S) - T_j(S) \\ &= \max(\phi_\mu + p_j - d_j, 0) + \max(t + p_j - d_i, 0) - \max(\phi_\mu + p_i - d_i, 0) - \max(t + p_j - d_j, 0) \\ &= \max(\phi_\mu + p_j - d_j, 0) + \max(t + p_j - d_i, 0) - \max(t + p_j - d_j, 0). \end{aligned}$$

If $t + p_j - d_i > 0$ then $t + p_j - d_j > 0$;

$$\begin{aligned} T(S') - T(S) &\leqslant \max(\phi_\mu + p_j - d_j, 0) + t + p_j - d_i - t - p_j + d_j = \max(\phi_\mu, d_j - p_j) + p_j - d_i \\ &\leqslant \max(\phi_\mu, d_j - p_j) - (d_i - p_i) \\ &\leqslant \max(\phi_\mu, d_j - p_j) - \min(d_i - p_i, \phi_{\mu'}) \leqslant 0. \end{aligned}$$

If $t + p_j - d_i \leqslant 0$ then $T(S') - T(S) \leqslant \max(\phi_\mu + p_j - d_j, 0) - \max(t + p_j - d_j, 0) \leqslant 0$.

Case 2: If job j is scheduled on a machine different from μ, the completion time and hence the tardiness of jobs initially scheduled after job j is increased at most by $(p_i - p_j)$. The number of such jobs is at most $(U^* - 1)$. The tardiness of jobs initially scheduled after i is decreased. Let t be the starting time of job j in S. Then

$$\begin{aligned} T(S') - T(S) &\leqslant (U^* - 1)(p_i - p_j) + \max(\phi_\mu + p_j - d_j, 0) + \max(t - d_i + p_i, 0) - \max(t + p_j - d_j, 0) \\ &\leqslant (U^* - 1)(p_i - p_j) + \max(\phi_\mu + p_j - d_j, 0) + \max(t - d_i + p_i, 0) - t - p_j + d_j \\ &\leqslant (U^* - 1)(p_i - p_j) + \max(\phi_\mu, d_j - p_j) - \min(0, d_i - p_i - t) - t \\ &\leqslant (U^* - 1)(p_i - p_j) + \max(\phi_\mu, d_j - p_j) - \min(t, d_i - p_i) \\ &\leqslant (U^* - 1)(p_i - p_j) + \max(\phi_\mu, d_j - p_j) - \min(\phi_{\mu'}, d_i - p_i). \end{aligned}$$

$T(S') - T(S) \leqslant 0$. □

Proof of Theorem 3.

Case 1: By construction we have

1.

$$\sum_{k \neq \mu} T_k(S') = \sum_{k \neq \mu} T_k(S),$$

2.

$$\sum_{h \in A_{i \to j}} \max(C_h(S') - d_h, 0) \leqslant \sum_{h \in A_{i \to j}} \max(C_h(S) - d_h, 0) + (p_j - p_i) \times \left|A_{i \to j}\right|$$

and

$$\sum_{h \in A_j} \max(C_h(S') - d_h, 0) \leqslant \sum_{h \in A_j} \max(C_h(S) - d_h, 0)$$

since $p_j \leqslant p_i$,

3.

$$\Delta_i(S') = \Delta_j(S) + (p_j - p_i)$$

and

4.

$$\phi_\mu + p_i \leqslant \phi_\mu + p_j \leqslant \Delta_j(S) + p_j.$$

Then $T(S') - T(S) \leqslant (p_j - p_i) \times \left|A_{i \to j}\right| + \Delta T3 + \Delta T4$ with $\Delta T3 = \max(\Delta_i(S') + p_i - d_i, 0) - \max(\phi_\mu + p_i - d_i, 0)$ we have $d_i \geqslant \phi_\mu + p_j \geqslant \phi_\mu + p_i$ then $\Delta T3 = \max(\Delta_i(S') + p_i - d_i, 0) = \max(\Delta_i(S) + p_j - d_i, 0)$ and $\Delta T4 = \max(\phi_\mu + p_j - d_j, 0) - \max(\Delta_j(S) + p_j - d_j, 0)$.

We have

Case 1.1: If $d_j \leqslant \phi_\mu + p_j \leqslant d_i \leqslant \Delta_j(S) + p_j$ then

$$T(S') - T(S) \leqslant (p_j - p_i) \times \left|A_{i \to j}\right| + (\phi_\mu + p_j - d_j) + (\Delta_j(S) + p_j - d_i) - (\Delta_j(S) + p_j - d_j)$$
$$\leqslant (p_j - p_i) \times (U - 2) - (d_i - (\phi_\mu + p_j)).$$

According to the conditions in the theorem, we have

$$(p_j - p_i) \times (U - 1) \leqslant d_i - (\phi_\mu + p_i)$$

which implies that

$$(p_j - p_i) \times (U - 1) - (p_j - p_i) \leqslant d_i - (\phi_\mu + p_i) - (p_j - p_i),$$

and we obtain

$$(p_j - p_i) \times (U - 2) - (d_i - (\phi_\mu + p_j)) \leqslant 0.$$

Case 1.2: If $d_j \leqslant \phi_\mu + p_j \leqslant \Delta_j(S) + p_j \leqslant d_i$ then

$$T(S') - T(S) \leqslant (p_j - p_i) \times \left|A_{i \to j}\right| + (\phi_\mu + p_j - d_j) - (\Delta_j(S) + p_j - d_j),$$
$$\leqslant (p_j - p_i) \times \left|A_{i \to j}\right| - (\Delta_j(S) - \phi_\mu).$$

The starting time of job j in S is equal to $\Delta_j(S) = \phi_\mu + p_i + \sum_{z \in A_{i \to j}} p_z$, then $T(S') - T(S) \leqslant (p_j - p_i) \times \left|A_{i \to j}\right| - (p_i + \sum_{z \in A_{i \to j}} p_z)$, we have also $(p_j - p_i) \times \left|A_{i \to j}\right| \leqslant \underline{p} \times \left|A_{i \to j}\right| \leqslant \sum_{z \in A_{i \to j}} p_z$.

Then we deduce for the previous cases that $\Omega(K, i)$ is dominated.

Case 2: For this case we have

1.
$$\sum_{k \notin \{\mu, m\}} T_k(S') = \sum_{k \notin \{\mu, m\}} T_k(S),$$

2.
$$\sum_{h \in A_i} \max(C_h(S') - d_h, 0) \leqslant \sum_{h \in A_i} \max(C_h(S) - d_h, 0) + (p_j - p_i) \times |A_i|$$

and

$$\sum_{h \in A_j} \max(C_h(S') - d_h, 0) \leqslant \sum_{h \in A_j} \max(C_h(S) - d_h, 0),$$

3. $|A_i| \leqslant U$,
4. $\Delta_i(S') = \Delta_j(S)$, and
5. $\phi_\mu + p_i \leqslant \phi_\mu + p_j \leqslant \Delta_j(S) + p_j$.

Then $T(S') - T(S) \leqslant (p_j - p_i) \times |A_i| + \Delta T3 + \Delta T4$ with $\Delta T3 = \max(\Delta_j(S) + p_i - d_i, 0) - \max(\phi_\mu + p_i - d_i, 0)$ and $\Delta T4 = \max(\phi_\mu + p_j - d_j, 0) - \max(\Delta_j(S) + p_j - d_j, 0)$.

Then the following cases:

Case 2.1: If $\Delta_j(S) + p_i \leqslant \phi_\mu + p_j$ and $d_j \leqslant \phi_\mu + p_j \leqslant d_i \leqslant \Delta_j(S) + p_j$ then $T(S') - T(S) \leqslant (p_i - p_j) \times U - (\Delta_j(S) - \phi_\mu)$,

Case 2.2: If $\phi_\mu + p_j \leqslant \Delta_j(S) + p_i$ we distinguish:

Case 2.2.1: If $d_i \leqslant \Delta_j(S) + p_i$ then $T(S') - T(S) \leqslant (p_i - p_j) \times U - (d_i - (\phi_\mu + p_i))$.

Case 2.2.2: If $\Delta_j(S) + p_i \leqslant d_i$ then $T(S') - T(S) \leqslant (p_i - p_j) \times U - (\Delta_j(S) - \phi_\mu)$.

According to conditions of the theorem we obtain for these various cases $T(S') - T(S) \leqslant 0$, therefore $\Omega(K, i)$ is dominated. □

Theorem 4. *For any partial schedule K and for any job i unscheduled in K, if there is another partial schedule K′ such that* $J(K') = J(K) \cup \{i\}$, $T(K') \leqslant T(K \circ i)$ *and* $[N - |J(K)| - 1] \times \Delta \leqslant T(K \circ i) - T(K')$, *then* $\Omega(K, i)$ *is dominated, where* $\Delta = \max_{k=1}^{M}(\varphi'_k - \varphi_k)$ *with* φ_k *and* φ'_k *being the kth smallest machine completion time in partial schedules* $K \circ i$ *and* K', *respectively.*

Proof of Theorem 4. Construct another schedule S from the partial schedule K' in the following way. For any k such that $1 \leqslant k \leqslant M$, identify from $K \circ i$ the machine finishing at the kth position, denoted g_k. The jobs scheduled after φ_k on machine g_k in $K \circ i$ are scheduled after φ'_k in the same order on the machine finishing at the kth position in partial schedule K', denoted as machine g'_k, i.e., $\varphi'_k \leqslant \varphi_k$ for any $k = 1, 2, \ldots, M$.

If $\Delta \leqslant 0$, it is obvious that $T(S) \leqslant T(\Omega(K, i))$ from the conditions defined in the theorem. Therefore, $\Omega(K, i)$ is dominated.

Consider now the case where $\Delta > 0$. Let $Q_k (k = 1, 2, \ldots, M)$ be the number of jobs scheduled on machine g'_k after φ'_k in S.

For any k such that $\varphi'_k - \varphi_k \leqslant 0$, the completion time of the jobs scheduled on g'_k after φ'_k in S is at most as large as in $\Omega(K, i)$. For any k such that $\varphi'_k - \varphi_k > 0$, the completion time of these Q_k jobs and hence the tardiness is increased by $\varphi'_k - \varphi_k$. Let $L = \{k / 1 \leqslant k \leqslant M \text{ and } \varphi_k < \varphi'_k\}$. Therefore

$$T(S) - T(\Omega(K, i)) \leqslant T(K') - T(K \circ i) + \sum_{k \in L} (\varphi'_k - \varphi_k) \times Q_k \leqslant T(K') - T(K \circ i) + \Delta \times \sum_{k \in L} Q_k.$$

Due to the fact that $\sum_{k \in L} Q_k \leqslant N - |J(K)| - 1$, $\Omega(K, i)$ is dominated. □

In what follows, we present the theoretical properties used by Azizoglu and Kircа [15] in their BAB method.

Proposition 1. *There exists an optimal schedule in which the sum of the processing time of the jobs processed on each machine does not exceed*

$$\frac{1}{M}\left\{\sum_{h=1}^{N} p_h + (M-1) \times \max_{1 \leqslant h \leqslant N}(p_h)\right\}.$$

Proposition 2. *There exists an optimal schedule in which job j is processed at final position on any one of the machines if*

$$d_j \geqslant \frac{1}{M}\left\{\sum_{h=1}^{N} p_h + (M-1) \times \max_{1 \leqslant h \leqslant N}(p_h)\right\}.$$

Proposition 3. *If all jobs have identical processing times then the EDD rule is optimal.*

Proposition 4. *If* $d_j \leqslant p_j$ *for* $j = 1 \ldots N$ *then the SPT rule is optimal.*

These properties are of course incorporated in our BAB algorithm. The combination of all these dominance properties and the lower and upper bounds, presented in the following paragraphs, allow to evaluate the intermediate solutions and then to eliminate the branches or the dominated solutions.

2.2. Lower bound

We devote this section to the lower bounding scheme we developed for the problem.

We note the difficulty in finding a good lower bound with usual relaxation schemes. Usually, a lower bound is obtained by relaxing the non preemption constraints. With this technique, it is impossible to obtain a lower bound in polynomial time since the same problem is NP-hard [12].

We obtain a lower bound by using a new method called multiple shortest processing time lists (MSPTL). This method proceeds in the following way.

Without loss of generality, we assume that jobs are numbered in the SPT order; namely, $p_1 \leqslant p_2 \leqslant \leqslant p_N$. We construct M single machine schedules in the following way. For the kth ($k = 1, 2, \ldots, M$) single machine schedule, jobs from k to N are scheduled in increasing order of their indexes. Let $C_{k,j}$, be the jth smallest completion time in the kth single machine schedule. Let G be the set of these completion times. The values of set G are then ranked in increasing order. Let $C_{\mathrm{MSPTL},[j]}$ be the jth element of the ranked set. Using the MSPTL method, a lower bound is obtained by associating the jth smallest due date to $C_{\mathrm{MSPTL},[j]}$.

Theorem 5. *If $(d_{|1|}, d_{|2|}, \ldots, d_{|N|})$ is the series obtained by sorting the series $(d_1, d_2, \ldots, d_N)$ in non decreasing order, the following relation holds*: $\sum_{j=1}^{N} \max(C_{\mathrm{MSPTL},[j]} - d_{|j|}, 0) \leqslant T^*$ *where T^* is the optimal total tardiness.*

This lower bound is based on the following Lemmas.

Lemma 1 (Chu 1992 [26]). *Let two series of numbers $(x_1, x_2, \ldots, x_N)$ and $(y_1, y_2, \ldots, y_N)$ and an ordered series $(x'_1, x'_2, \ldots, x'_N)$ be so that 1/ $x'_1 \leqslant x'_2 \leqslant \cdots \leqslant x'_N$, 2/ for $j = 1, \ldots, N$, $x'_j \leqslant x_j$. If $(y_1, y_2, \ldots, y'_N)$ is the series obtained by sorting the series $(y_1, y_2, \ldots, y_N)$ in non decreasing order, the following relation holds*:

$$\sum_{j=1}^{N} \max(x'_j - y'_j, 0) \leqslant \sum_{j=1}^{N} \max(x_j - y_j, 0).$$

Lemma 2. *Let S and S^{SPT} be any feasible single machine schedule and the schedule obtained by scheduling jobs in their indexes on a single machine. Then $C_{[j]}(S^{\mathrm{SPT}}) \leqslant C_{[j]}(S)$, $j = 1, \ldots, N$, where $C_{[j]}(S^{\mathrm{SPT}})$ (respectively $C_{[j]}(S)$) is the jth smallest completion time with the solution S^{SPT} (respectively S).*

Proof. For the proof, see Conway et al. [29], Baker [30] or Chu [28]. □

Proof of Theorem 5. Let S be any feasible solution to the problem considered, and $C_{m,[j]}(S)$ the jth completion time on machine m in S. Let I_m be the smallest indexed job scheduled on machine m ($m = 1, \ldots, M$) in S. Without loss of generality, we renumber the machines in increasing order of I_m value. We obtain then $I_1 \leqslant I_2 \leqslant \cdots \leqslant I_M$.

Let S_m be the sub-schedule on machine m in S. Construct another sub-schedule, noted S_m^{SPT} by ordering jobs in S_m in SPT order. According to the Lemma 2, we have $C_{m,[j]}(S_m^{\mathrm{SPT}}) \leqslant C_{m,[j]}(S_m)$ for $m = 1, \ldots, M$ and $j = 1, \ldots, |S_m|$. Note that $p_{[j]}^{(S_m^{\mathrm{SPT}})} \geqslant p_{m,[j]}^{(S_{\mathrm{MSPTL}})}$ with $p_{[j]}^{(S_m^{\mathrm{SPT}})}$ (respectively $p_{m,[j]}^{(S_{\mathrm{MSPTL}})}$) being the processing time of the job scheduled on machine m in the jth position with solution S_m^{SPT} (respectively S_{MSPTL}). Therefore

$$C_{m,[j]}(S_m^{\mathrm{SPT}}) = \sum_{k=1}^{j} p_{[j]}^{(S_m^{\mathrm{SPT}})} \geqslant \sum_{k=1}^{j} p_{m,[j]}^{(S_{\mathrm{MSPTL}})} = C_{m,[j]}(S_{\mathrm{MSPTL}}).$$

We can deduce that

$$C_{m,[j]}(S^{\mathrm{SPT}}) \geqslant C_{m,[j]}(S_{\mathrm{MSPTL}}) \text{for } m = 1, \ldots, M \text{ and } j = 1, \ldots, N. \tag{3}$$

Let $G(S) = \{C_{m,[j]}(S), \forall m, \forall j\}$ be the set of the completion times according to the schedule S and $G(S_{\mathrm{MSPTL}}) = \{C_{m,[j]}(S_{\mathrm{MSPTL}}), \forall m, \forall j\}$ the set of the completion times obtained with the solution S_{MSPTL}. Let $C_{[j]}(S)$ (respectively $C_{[j]}(S_{\mathrm{MSPTL}})$) be the jth smallest value of the set $G(S)$ (respectively $G(S_{\mathrm{MSPTL}})$). From the relation (3), we can deduce for $j = 1, \ldots, N$, $C_{[j]}(S_{\mathrm{MSPTL}}) \leqslant C_{[j]}(S)$.

From the previous results, we know that for $j = 1 \ldots N$, $C_{[j]}(S_{\mathrm{MSPTL}}) \leqslant C_{[j]}(S^*)$ and we have the series $(d_{|1|}, d_{|2|}, \ldots, d_{|N|})$ which is a sorted series of the jobs due dates $(d_1, d_2, \ldots, d_N)$. According to Lemma 1, we obtain the following relation:

$$\sum_{j=1}^{N} \max(C_{[j]}(S_{\mathrm{MSPTL}}) - d_{[j]}, 0) \leqslant \sum_{j=1}^{N} \max(C_{[j]}(S^*) - d_j, 0),$$

which implies that

$$\sum_{j=1}^{N} \max(C_{[j]}(S_{\mathrm{MSPTL}}) - d_{[j]}, 0) \leqslant T^*(P). \quad \square$$

In the BAB algorithm of Azizoglu and Kirca [15], the lower bound used is based on the work of Kondakci et al. [31]. Kondakci et al. [31] considered that the jobs are indexed in the SPT order and proposed a lower bound equal to $\sum_{j=1}^{N} \max\{(\sum_{h=1}^{j} p_h/M) - d_{[j]}, 0\}$. Azizoglu and Kirca, by extending this last result to the case of parallel machines, proposed for a given partial schedule K the following lower bound

$$\sum_{j \notin J(K)} \max\left\{\sum_{j=1}^{N}\left\{\phi_{\mu}(K) + \frac{p_h}{M}\right\} - d_{|j|}, 0\right\},$$

where $d_{[j]}$ is the jth smallest due date of the a job unscheduled by K.

This lower bound is equivalent to allowing job splitting. From ϕ_k, the job with the shortest processing time is split into M identical fractions, each fraction on a machine. The same process is repeated until all jobs are scheduled. This lower bound can be improved with a slight modification. Instead of splitting the job with the shortest processing time to M fractions, it is split into as many fractions as the number of available machine.

We use for our algorithm a lower bound based on the combination of the two lower bounds previously quoted with $\mathrm{LB} = \max(\mathrm{LB1}, \mathrm{LB2})$ with LB1 the lower bound obtained with the method used by Azizoglu and Kirca and LB2 the lower bound that we have developed.

2.3. Upper bound

The upper bound is generated by applying a heuristic which consists in choosing the best solution obtained among those obtained with priority rules: SPT, EDD, minimum slack. This upper bound generation technique is an application of the method suggested by Dogramaci and Surkis [7].

3. Branch-and-bound algorithm

The BAB algorithm developed is inspired by the BAB algorithm that we proposed to solve a parallel machine problem to minimize total completion time [26,27]. The algorithm proceeds in the following way. Each node represents a partial schedule. An initial solution is first calculated which provides the first upper

bound. This first solution is obtained, as indicated above, by choosing the best solution resulting from the SPT, EDD and minimum slack rules. The branching consists of appending a new job after a partial schedule; i.e. on the machine available the earliest in the partial schedule. For this purpose, a list of unexplored nodes are arranged in increasing order of their lower bounds, with ties broken by non increasing number of scheduled jobs. For each node, the machines are sorted in non decreasing order of their availability time. Before the creation of a new node, a series of dominance properties are checked (given by the Theorems 2–4 and different proposition above). For each node of the search tree, which are not eliminated by the dominance properties, a lower bound is calculated. If the value of the lower bound

Table 1
Problem groups

TF	RDD				
	0.2	0.4	0.6	0.8	1.0
0.2	1	2	3	4	5
0.4	6	7	8	9	10
0.6	11	12	13	14	15
0.8	16	17	18	19	20
1.0	21	22	23	24	25

Table 2
Performance of the branch-and-bound algorithms for $N = 15$ and $M = 2$

Gp	Bab YC1			Bab AK			Bab YC2			Bab YC3		
	Time	Node	T (%)	Time	Node	T (%)	Time	Node	T (%)	Time	Node	T (%)
1	209.44	9,635.40	100	—	—	0	221.52	9,722.80	100	1,455.05	1,663,574.00	20
2	86.33	3,816.00	100	1,548.89	2,749,219.00	60	90.56	3,820.20	100	1,210.08	1,338,893.00	40
3	186.33	6,261.60	100	949.61	1,366,023.67	60	197.33	6,272.40	100	1,091.06	1,444,903.00	40
4	271.59	9,228.20	100	1,330.54	793,379.20	100	314.83	10,185.40	100	481.39	583,119.60	100
5	95.60	6,076.80	100	1,521.26	432,722.20	100	104.35	6,214.20	100	260.09	162,537.40	100
6	58.86	6,214.20	100	823.92	165,880.00	60	63.37	6,336.20	100	901.30	1,047,053.33	60
7	160.59	8,776.80	100	1,282.54	2,546,424.00	40	192.56	9,227.60	100	851.36	1,089,917.33	60
8	146.90	9,127.60	100	1,068.27	1,231,336.00	60	175.85	9,797.60	100	1,099.20	959,001.00	60
9	208.73	10,871.20	100	456.81	591,729.75	80	258.16	11,564.40	100	427.61	453,514.00	80
10	62.86	4,542.40	100	366.89	703,764.50	80	76.44	4,929.00	100	366.79	399,261.75	80
11	144.58	12,789.40	100	831.61	487,200.40	100	165.36	13,271.60	100	275.31	161,111.80	100
12	49.09	6,626.60	100	1,415.06	2,137,008.67	60	58.32	7,216.60	100	552.96	456,349.25	80
13	315.06	14,899.20	100	1,978.95	1,125,365.80	100	395.99	15,914.20	100	589.26	513,752.75	80
14	255.38	13,725.60	100	1,885.41	1,103,253.80	100	365.27	15,271.40	100	1,023.98	844,159.33	60
15	65.65	7,591.20	100	726.56	1,035,465.25	80	94.93	8,384.60	100	749.34	487,199.25	80
16	1.01	743.00	100	41.06	62,423.40	100	1.10	761.20	100	2.00	1,436.80	100
17	2.23	1,408.60	100	69.19	179,527.60	100	2.78	1,488.60	100	4.23	5,034.00	100
18	18.63	4,639.80	100	364.95	291,152.80	100	21.19	4,930.20	100	42.53	20,696.20	100
19	42.80	6,074.40	100	370.65	407,360.20	100	53.15	6,536.80	100	158.57	32,166.40	100
20	53.19	7,850.00	100	559.38	693,971.50	80	71.01	8,755.00	100	471.76	432,337.25	80
21	0.02	27.60	100	9.84	13,411.40	100	0.02	28.00	100	0.04	43.00	100
22	0.16	197.60	100	25.14	54,328.40	100	0.16	197.60	100	0.35	470.00	100
23	3.58	1,734.40	100	88.60	99,535.60	100	4.44	1,921.00	100	7.21	7,296.80	100
24	45.42	5,821.60	100	235.50	207,086.80	100	58.79	6,379.20	100	129.01	25,541.20	100
25	14.39	3,585.60	100	360.70	346,753.00	100	19.60	4,037.20	100	55.94	19,410.00	100

Table 3
Performance of the branch-and-bound algorithms for $N = 15$ and $M = 3$

Gp	Bab YC1			Bab AK			Bab YC2			Bab YC3		
	Time	Node	T (%)	Time	Node	T (%)	Time	Node	T (%)	Time	Node	T (%)
1	544.05	12,326.20	100	—	—	0	562.22	12,359.80	100	1,488.65	1,612,865.20	20
2	143.78	4,912.80	100	—	—	0	147.49	4,934.00	100	—	—	0
3	314.52	7,050.20	100	1,362.06	1,789,575.60	40	329.51	7,128.60	100	1,514.15	1,730,732.60	20
4	302.96	6,432.60	100	365.95	565,994.00	80	330.43	6,688.80	100	710.55	98,900.80	100
5	78.62	3,767.60	100	1,556.47	700,307.60	100	85.60	3,847.00	100	1,562.20	140,944.80	100
6	126.61	6,975.80	100	1,477.10	2,336,254.80	40	129.96	7,013.20	100	1,158.40	1,246,674.40	40
7	387.40	10,816.00	100	1,472.10	1,799,446.00	20	407.17	10,932.00	100	1,170	1,311,292.60	40
8	360.35	11,720.00	100	1,466.20	1,760,123.40	20	385.19	12,173.60	100	1,254.14	1,272,582.50	40
9	247.18	14,945.40	100	1,232.56	1,410,190.20	40	398.57	402,588.40	80	783.52	954,645.60	60
10	288.77	7,790.80	100	778.95	882,846.80	60	299.34	7,835.20	100	409.64	459,873.80	80
11	524.11	20,508.20	100	1,496.06	1,632,131.50	40	457.87	21,478.80	100	866.99	844,000.00	60
12	344.57	13,192.40	100	—	—	0	345.56	13,436.80	100	1,188.28	124,096.40	60
13	761.33	20,601.40	100	1,409.04	1,676,097.80	40	724.48	25,005.80	100	1,065.41	895,027.20	60
14	514.43	23,076.60	100	1,185.08	1,663,976.00	40	518.74	23,084.20	100	1,094.43	1,217,263.40	40
15	288.98	11,125.00	100	1,519.02	2,010,772.20	20	330.94	11,466.60	100	860.28	940,434.00	60
16	4.40	1,175.40	100	252.18	338,500.80	100	4.63	1,188.00	100	7.12	6,418.80	100
17	9.54	2,102.80	100	701.14	730,177.80	100	10.35	2,109.60	100	21.48	21,505.60	100
18	108.07	7,683.80	100	1,388.98	1,482,613.80	60	113.17	7,800.40	100	622.30	460,020.80	80
19	263.44	10,289.00	100	777.75	1,757,907.00	80	283.97	10,593.80	100	451.28	531,150.20	80
20	343.63	13,859.20	100	1,321.06	1,612,703.40	40	371.91	14,313.20	100	835.13	853,771.60	60
21	0.01	53.20	100	903.67	293,007.80	100	0.01	53.20	100	3.10	117.40	100
22	0.47	349.60	100	1,333.56	421,354.00	100	0.53	349.60	100	2.50	1,555.00	100
23	25.33	3,269.20	100	820.94	905,821.30	80	30.99	3,537.60	100	51.18	54,421.60	100
24	406.49	12,277.00	100	867.63	7,380,961.80	80	493.00	13,124.00	100	450.41	90,799.60	100
25	125.77	7,364.40	100	898.22	1,052,561.80	80	150.39	7,930.80	100	549.99	450,881.50	80

obtained is greater than or equal to the value of the current solution or the value of the upper bound, this node is eliminated.

4. Computational results

This section describes the computational results. The BAB algorithm was programmed and tested on an HP workstation using C compiler. The test examples were generated using the scheme proposed by Koulamas [1] or Azizoglu and Kirca [15] for the same problem. We note that this scheme of generation is an adaptation of the scheme proposed by Potts and Van Wassenhove [10] for single machine problem. Test examples were generated with $M = 2, 3, or\, 4$ machines and $N = 15, 20, or\, 30$ jobs. We generate, for each job, an integer processing time p_i using a uniform distribution [1,100] and an integer due date in the interval $[P\times(1 - \mathrm{TF} - (\mathrm{RDD}/2)), P\times(1 - \mathrm{TF} + (\mathrm{RDD}/2))]$, where $P = (\sum_{j=1}^{N} p_j/M)$; $\mathrm{TF} \in \{0.2, 0.4, 0.6, 0.8, 1.0\}$ and $\mathrm{RDD} \in \{0.2, 0.4, 0.6, 0.8, 1.0\}$. We generate 125 instances for each pair of (N, M) with 5 problems for each pair (TF, RDD).

For each combination (N, M), the 125 problems are divided into 25 representative groups with all possible combinations of the two parameters TF and RDD as presented in Table 1. The columns represent

Table 4
Performance of the branch-and-bound algorithms for $N = 15$ and $M = 4$

Gp	Bab YC1			Bab AK			Bab YC2			Bab YC3		
	Time	Node	T (%)	Time	Node	T (%)	Time	Node	T (%)	Time	Node	T (%)
1	934.04	19,914.80	100	—	—	—	976.10	19,910.60	100	—	—	—
2	257.46	6,344.20	100	—	—	—	273.24	6,370.00	100	—	—	—
3	578.75	9,218.60	100	—	—	—	621.68	9,228.80	100	—	—	—
4	400.37	7,507.20	100	—	—	—	423.23	7,645.40	100	—	—	—
5	48.95	2,291.00	100	—	—	—	52.49	2,294.20	100	—	—	—
6	510.03	11,241.40	100	—	—	—	553.88	11,293.40	100	—	—	—
7	617.67	11,541.20	100	—	—	—	619.80	11,568.40	100	—	—	—
8	658.52	13,111.80	100	—	—	—	682.61	13,354.20	100	—	—	—
9	498.82	403,514.40	80	—	—	—	500.33	403,538.60	80	—	—	—
10	93.10	12,050.20	100	—	—	—	111.79	12,482.80	100	—	—	—
11	968.21	413,135.60	80	—	—	—	974.39	413,164.80	80	—	—	—
12	442.54	13,216.80	100	—	—	—	458.19	13,478.00	100	—	—	—
13	735.67	413,980.00	80	—	—	—	739.30	414,055.40	80	—	—	—
14	699.26	412,445.40	80	—	—	—	842.88	802,977.60	60	—	—	—
15	305.98	14,531.60	100	—	—	—	473.92	14,813.80	100	—	—	—
16	30.99	2,171.40	100	—	—	—	33.08	2,171.00	100	—	—	—
17	9.60	1,728.80	100	—	—	—	9.73	1,729.80	100	—	—	—
18	215.12	8,504.20	100	—	—	—	219.42	8,579.40	100	—	—	—
19	440.91	12,353.60	100	—	—	—	524.64	12,584.80	100	—	—	—
20	723.56	16,154.00	100	—	—	—	729.07	16,092.20	100	—	—	—
21	0.20	129.80	100	—	—	—	0.20	129.80	100	—	—	—
22	1.09	487.00	100	—	—	—	1.10	487.00	100	—	—	—
23	71.72	3,439.40	100	—	—	—	75.35	3,594.60	100	—	—	—
24	455.55	10,478.60	100	—	—	—	675.44	12,279.20	100	—	—	—
25	344.36	8,954.60	100	—	—	—	389.36	9,629.20	100	—	—	—

the values of RDD, the rows represent the values of TF and each cell of the table gives the corresponding group number.

The computational results are given by Tables 2–6 where "Node", "Time" and "T" represent, respectively, the average number of nodes generated, the average computation time in CPU seconds and the percentage of problems solved over 5 problems of the corresponding class in a period of 30 minutes.

We applied to the various problems four different algorithms. We have:

Bab YC1: This algorithm takes into account all the dominance properties, the upper and lower bounding schemes presented in previous section.
Bab AK: the algorithm proposed by Azizoglu and Kirca [15].
Bab YC2: which is identical to Bab YC1 except that only the property given by Theorem 4 is considered.
Bab YC3: which is identical to BabYC1 except that the property given by Theorem 4 is not used.

The two algorithms Bab YC2 and Bab YC3 are tested to check the effectiveness of the dominance property given by Theorem 4.

With our BAB algorithm, all problems with 15 jobs and 2 or 3 machines are solved while the method of Azizoglu and Kirca [15] failed to solve 20% of the problems with 15 jobs and 2 machines and 50% of the problems with 15 jobs and 3 machines.

We also obtained solutions for 97% of the problems for ($N = 15, M = 4$) and practically 50% of the problems for ($N = 20, M = 2$).

The method that we developed could solve problems with up to 30 jobs and 2 machines but only problems of group 21 i.e. TF = 1.0 and RDD = 0.2 are all solved. As we noted, the method of Azizoglu and Kirca failed to solve problems with ($N = 15, M = 4$), ($N = 20, M = 2$) and ($N = 30, M = 2$) within the specified time period. This clearly shows the superiority of our BAB algorithm. We also note, through the various experiments, the effectiveness of the property given by Theorem 4. Except rare case, it makes it possible to eliminate a large number of nodes in the search tree and leads quickly to an optimal solution. We note also the influence of the TF parameters and RDD on the performances of the exposed methods of resolution.

Table 5
Performance of the branch-and-bound algorithms for $N = 20$ and $M = 2$

Gp	Bab YC1			Bab AK			Bab YC2			Bab YC3		
	Time	Node	T (%)	Time	Node	T (%)	Time	Node	T (%)	Time	Node	T (%)
1	1,518.17	1,602,890.60	20	—	—	—	1,520.50	1,602,890.60	20	—	—	—
2	—	—	0	—	—	—	—	—	0	—	—	—
3	1,594.56	1,604,074.20	20	—	—	—	1,594.56	1,604,074.20	20	—	—	—
4	1,288.16	1,208,176.40	40	—	—	—	1,324.23	1,208,991.20	40	—	—	—
5	1,021.62	435,559.40	80	—	—	—	1,049.54	435,879.40	80	—	—	—
6	—	—	0	—	—	—	—	—	0	—	—	—
7	1,447.81	1,206,807.00	40	—	—	—	1,473.46	1,206,914.20	40	—	—	—
8	1,082.77	1,200,861.00	40	—	—	—	1,083.21	1,200,932.60	40	—	—	—
9	1,108.97	817,179.60	60	—	—	—	1,043.84	817,748.80	60	—	—	—
10	577.25	416,058.40	80	—	—	—	669.48	417,978.80	80	—	—	—
11	1,470.25	1,602,584.20	20	—	—	—	1,472.50	1,602,615.60	20	—	—	—
12	—	—	0	—	—	—	—	—	0	—	—	—
13	—	—	0	—	—	—	—	—	0	—	—	—
14	1,538.25	1,607,179.40	20	—	—	—	1,566.41	1,608,065.80	20	—	—	—
15	1,259.86	826,397.60	60	—	—	—	1,390.19	1,212,045.20	40	—	—	—
16	762.51	28,201.60	100	—	—	—	782.12	29,513.20	100	—	—	—
17	924.30	810,910.80	60	—	—	—	982.22	811,829.20	60	—	—	—
18	1,083.08	1,200,889.00	40	—	—	—	1,083.28	1,200,905.00	40	—	—	—
19	1,564.32	1,604,466.00	20	—	—	—	1,632.02	1,605,138.20	20	—	—	—
20	1,482.16	456,272.40	80	—	—	—	1,259.92	462,723.40	80	—	—	—
21	0.57	334.80	100	—	—	—	0.69	380.20	100	—	—	—
22	42.10	7,002.40	100	—	—	—	56.94	7,531.20	100	—	—	—
23	748.23	57,697.60	100	—	—	—	855.26	59,014.00	100	—	—	—
24	775.30	842,824.60	60	—	—	—	790.48	843,172.80	60	—	—	—
25	1,027.34	813,227.20	60	—	—	—	1,119.62	814,742.00	60	—	—	—

Table 6
Performance of the branch-and-bound algorithms for $N = 30$ and $M = 2$

Gp	Bab YC1			Bab AK			Bab YC2			Bab YC3		
	Time	Node	T (%)	Time	Node	T (%)	Time	Node	T (%)	Time	Node	T (%)
21	116.26	9,377.00	100	—	—	—	125.77	9,971.75	100	—	—	—

5. Conclusions

This paper addressed the identical parallel machine scheduling problem to minimize total tardiness. We have developed dominance properties, lower and upper bounding schemes incorporated into a branch-and-bound algorithm. Computational experiments show that we can obtain optimal solutions in some cases with 30 jobs and 2 machines within a reasonable amount of computation time. Our algorithm is shown to be more efficient than existing methods in the literature.

References

[1] C. Koulamas, The total tardiness problem: review and extensions, Operational Research 42 (1994) 1025–1041.
[2] J. Du, J.Y.T. Leung, Minimizing total tardiness on one machine is NP-hard, Mathematics of Operational Research 15 (1990) 483–495.
[3] J.K. Lenstra, A.H.G. Rinnooy Kan, P. Brucker, Complexity of machine scheduling problems, Annals of Discrete Mathematics 1 (1997) 343–362.
[4] E.R. Montagne Jr., Sequencing with time delay costs, Arizona State University, Industrial Engineering Research Bulletin 5 (1969) 20–31.
[5] L.J. Wilkerson, J.D. Irwin, An improved algorithm for scheduling independent jobs, AIIE Transactions 3 (1971) 239–245.
[6] K.R. Baker, Procedure for sequencing jobs with one resource type, International Journal of Production Researches 11 (1998) 125–138.
[7] A. Dogramaci, J. Surkis, Evaluation of a heuristic for scheduling independent jobs on parallel identical processors, Management Science 25 (1979) 1208–1216.
[8] J.C. Ho, Y.-L. Chang, Heuristics for minimizing mean tardiness for *m* parallel machines, Naval Research Logistics 38 (1991) 367–381.
[9] S.S. Panwalker, M.L. Smith, C. Koulamas, A heuristic for the single machine total tardiness problem, European Journal of Operational Research 70 (1993) 304–310.
[10] C.N. Potts, L.N. Van Wassenove, Single machine tardiness sequencing heuristics, IIE Transactions 23 (1991) 346–354.
[11] C. Koulamas, Decomposition and hybrid simulated annealing heuristics for the parallel-machine total tardiness problem, Naval Research Logistics 44 (1997) 109–125.
[12] K. Chen, J.S. Wong, J.C. Ho, A heuristic algorithm to minimize tardiness for parallel machines, Proceedings of ISMM\International Conference. ISSM-ACTA Press, Anaheim, CA, USA, 1997, pp, 118–121.
[13] B. Alidaee, D. Rosa, Scheduling parallel machines to minimize total weighted and un-weighted tardiness, Computers Operational Research 24 (8) (1997) 775–788.
[14] K.R. Baker, J.W. Bertrand, A dynamic priority rule for sequencing against due-date, Journal of Operational Management 3 (1982) 37–42.
[15] M. Azizoglu, O. Kirca, Tardiness minimization on parallel machines, International Journal of Production Economics 55 (1998) 163–168.
[16] E.L. Lawler, On scheduling problems with deferral costs, Management Sciences 11 (1964) 280–288.
[17] J.G. Root, Scheduling with deadlines and loss functions on k parallel machines, Management Sciences 11 (1965) 460–475.
[18] A.A.B. Pritsker, L.J. Walters, P.M. Wolf, Multi-project scheduling with limited resources: A zero–one programming approach, Management Sciences 16 (1969) 93–108.
[19] S.E. Elmaghraby, S.H. Park, Scheduling jobs on a number of identical machines, AIIE Transactions 6 (1974) 1–13.
[20] J.W. Barnes, J.J. Brennan, An improved algorithm for independent jobs to reduce mean finishing time, AIIE Transactions 17 (1977) 382–387.
[21] E.M. Arkin, R.O. Roundy, Weighted-tardiness scheduling on parallel machines with proportional weights, Operations Research 39 (11) (1991) 64–81.
[22] H. Emmons, M. Pinedo, Scheduling stochastic jobs with due dates on parallel machines, European Journal of Operational Research 47 (1990) 49–55.
[23] A. Guinet, Scheduling independent jobs on uniform parallel machines to minimize tardiness criteria, Journal of Intelligent Manufacturing 6 (1995) 95–103.
[24] H. Emmons, Scheduling to a common due date on parallel uniform processors, Naval Research Logistics 34 (1987) 803–810.
[25] M. Azizoglu, O. Kirca, Scheduling jobs on unrelated parallel machines to minimize regular cost functions, IIE Transactions 31 (1999) 153–159.

[26] F. Yalaoui, C. Chu, Parallel machine scheduling to minimize total completion time with release dates constraints, CD-ROM of MCPL'2000, Grenoble, France, July 5–7, 2000.
[27] F. Yalaoui, C. Chu, Parallel machine scheduling to minimize total completion time with release dates constraints: An exact method, Discrete Applied Mathematics Journal. Submitted for publication.
[28] C. Chu, A branch-and-bound algorithm to minimize total tardiness with different release dates, Naval Research Logistics 39 (1992) 265–283.
[29] R.W. Conway, W.L. Maxwell, L.W. Miller, Theory of Scheduling, Addison-Wesley, Reading, MA, 1967.
[30] K.R. Baker, Introduction to Sequencing and Scheduling, Wiley, New York, 1974.
[31] S. Kondakci, O. Kirca, M. Azizoglu, An efficient algorithm for single machine tardiness problem, International Journal of Production Economics 36 (1994) 213–219.